

Java OOP — Encapsulation

Beginner-friendly notes with getters, setters, validation, interview points, and practice.

1. What is Encapsulation?

Encapsulation means wrapping data and methods together inside a class and controlling direct access to that data. The common approach is **private variables + public getters/setters**.

2. Why Do We Need Encapsulation?

If important data is public, outside code can change it to invalid values. Encapsulation protects the data and lets the class control how values are read or changed.

```
class BankAccount {
    double balance;
}

BankAccount b = new BankAccount();
b.balance = -50000; // unsafe
```

3. What is private?

private is an access modifier. A private variable can be accessed only inside the same class.

```
class Student {
    private int marks;

    void setInsideClass() {
        marks = 80; // allowed
    }
}

// s.marks = 80; // not allowed
```

4. Getter and Setter

A **getter** is generally used to read a value. A **setter** is generally used to modify a value.

```
class Student {
    private int marks;

    public void setMarks(int marks) {
        this.marks = marks;
    }

    public int getMarks() {
        return marks;
    }
}

Student s = new Student();
s.setMarks(80);
System.out.println(s.getMarks());
```

5. Complete Student Example

```
import java.util.*;

class Student {
    private String name;
    private int marks;
```

```

    public void setName(String name) {
        this.name = name;
    }

    public void setMarks(int marks) {
        this.marks = marks;
    }

    public String getName() {
        return name;
    }

    public int getMarks() {
        return marks;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
    }
}

public class EncapsulationExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Student s1 = new Student();

        String name = sc.nextLine();
        int marks = sc.nextInt();

        s1.setName(name);
        s1.setMarks(marks);

        s1.display();
    }
}

```

6. Validation with a Setter

A setter can validate data before storing it. For example, marks can be restricted to the range 0–100.

```

    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100) {
            this.marks = marks;
        } else {
            System.out.println("Invalid marks");
        }
    }
}

```

7. Real-World Bank Account

A bank balance should not be directly changed from outside the class. Controlled methods such as deposit() can update it.

```

class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

```

```
}  
}
```

8. Encapsulation Structure

```
class Employee {  
    private String name;  
    private double salary;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setSalary(double salary) {  
        this.salary = salary;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
}
```

9. Encapsulation vs this

this refers to the current object. **Encapsulation** controls access to object data. They are often used together.

10. Important Interview Question

What is Encapsulation?

Encapsulation is the process of bundling data and methods inside a class and restricting direct access to the data using access control, commonly through private variables and public getters/setters.

11. Logic Building

When a problem says data should not be directly accessible from outside the class, think: **private variable + getter/setter**. When a value must be validated before storage, put the validation inside the setter.

```
private double salary;  
  
public void setSalary(double salary) {  
    if (salary >= 0) {  
        this.salary = salary;  
    } else {  
        System.out.println("Invalid salary");  
    }  
}
```

12. Practice Problem

Create a class **Employee** with private variables **name** and **salary**. Create `setName()`, `getName()`, `setSalary()`, and `getSalary()`. Salary must not be negative. Use Scanner, create an Employee object, use setters to store values, and getters to display them.

13. Key Takeaways

- Encapsulation protects object data.
- Use **private** for fields that should not be directly accessed.
- Use getters to read private data.
- Use setters to modify private data.
- Validation can be placed inside setters.
- **this** is commonly used inside setters to distinguish fields from parameters.
- Encapsulation is one of the four major OOP pillars.